WEEK 8: DATABASE RELATIONSHIP ANALYSIS USING JOINS

DBMS Project – Pet Shop Management System

Scenario

The Pet Shop Management System stores customer, product, order, order-detail, and payment information in separate tables. SQL joins are used to combine these related tables to study customer purchases, order details, product sales, and payment status.

Tables and Columns Used

CUSTOMER(CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE, CREATED_AT)
ORDERS(ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS)
ORDERDETAILS(ORDER_DETAILS_ID, ORDER_ID, PRODUCT_ID, QUANTITY, PRICE)
PRODUCT(PRODUCT_ID, PRODUCT_NAME, CATEGORY_ID, PRICE, STOCK_QUANTITY)
PAYMENT(PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)

The SQL below uses the column names visible in the screenshots. If a query reports an invalid identifier, check the column name with DESC table_name; before changing the query.

1. Implement INNER JOIN

Concept: INNER JOIN returns only records that have matching values in both tables.

1.1 Display customer details along with their orders

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME,
  4      C.LAST_NAME,
  5      O.ORDER_ID,
  6      O.ORDER_DATE,
  7      O.TOTAL_AMOUNT,
  8      O.ORDER_STATUS
  9  FROM CUSTOMER C
 10  INNER JOIN ORDERS O
 11  ON C.CUSTOMER_ID =
 12     CASE O.CUSTOMER_ID
 13         WHEN 'C001' THEN 'S101'
 14         WHEN 'C002' THEN 'S102'
 15         WHEN 'C003' THEN 'S103'
 16         WHEN 'C004' THEN 'S104'
 17         WHEN 'C005' THEN 'S105'
 18     END;

CUSTOMER_ID  FIRST_NAME      LAST_NAME       ORDER_ID   ORDER_DATE      TOTAL_AMOUNT ORDER_STATUS
------------ --------------- --------------- ---------- --------------- ------------ ---------------
S101         Akshaya         Nivasini        O001       01-SEP-26            1200.00 Pending
S102         Prithish        Raj             O002       03-SEP-26             850.00 Shipped
S103         kshaunish       Rajesh          O003       05-SEP-26            1500.00 Delivered
S104         kaniska         Ram             O004       07-SEP-26             650.00 Pending
S105         Rukmani         Kuppusamy       O005       10-SEP-26            2100.00 Delivered
S101         Akshaya         Nivasini        O006       12-SEP-26            1000.00 Pending
S102         Prithish        Raj             O007       13-SEP-26            1200.00 Shipped
S103         kshaunish       Rajesh          O008       14-SEP-26             850.00 Delivered
S104         kaniska         Ram             O009       15-SEP-26            1500.00 Pending
S105         Rukmani         Kuppusamy       O010       16-SEP-26             650.00 Shipped

10 rows selected.
```

1.2 Display order details with payment information

```
SQL> SELECT
  2      O.ORDER_ID,
  3      O.ORDER_DATE,
  4      O.TOTAL_AMOUNT,
  5      O.ORDER_STATUS,
  6      P.PAYMENT_ID,
  7      P.PAYMENT_MODE,
  8      P.PAYMENT_STATUS,
  9      P.TRANSACTION_AMOUNT
 10  FROM ORDERS O
 11  INNER JOIN PAYMENT P
 12  ON O.ORDER_ID = P.ORDER_ID;

ORDER_ID   ORDER_DATE      TOTAL_AMOUNT ORDER_STATUS    PAYMENT_ID   PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
---------- --------------- ------------ --------------- ------------ --------------- --------------- ------------------
O001       01-SEP-26            1200.00 Pending         PAY001       UPI             Success                       2500
O002       03-SEP-26             850.00 Shipped         PAY002       Card            Success                       1800
O003       05-SEP-26            1500.00 Delivered       PAY003       Cash            Success                       1200
O004       07-SEP-26             650.00 Pending         PAY004       UPI             Failed                        3000
O005       10-SEP-26            2100.00 Delivered       PAY005       Card            Success                       2200
O006       12-SEP-26            1000.00 Pending         PAY006       UPI             Success                       1500
O007       13-SEP-26            1200.00 Shipped         PAY007       Cash            Failed                         900
O008       14-SEP-26             850.00 Delivered       PAY008       UPI             Success                       3500
O009       15-SEP-26            1500.00 Pending         PAY009       Card            Success                       2800
O010       16-SEP-26             650.00 Shipped         PAY010       UPI             Pending                       1700

10 rows selected.
```

1.3 Retrieve products purchased by customers

```
SQL> SELECT
  2      C.FIRST_NAME,
  3      C.LAST_NAME,
  4      PR.PRODUCT_NAME,
  5      OD.QUANTITY
  6  FROM CUSTOMER C
  7  INNER JOIN ORDERS O
  8      ON C.CUSTOMER_ID =
  9         CASE O.CUSTOMER_ID
 10             WHEN 'C001' THEN 'S101'
 11             WHEN 'C002' THEN 'S102'
 12             WHEN 'C003' THEN 'S103'
 13             WHEN 'C004' THEN 'S104'
 14             WHEN 'C005' THEN 'S105'
 15         END
 16  INNER JOIN ORDERDETAILS OD
 17      ON O.ORDER_ID = OD.ORDER_ID
 18  INNER JOIN PRODUCT PR
 19      ON OD.PRODUCT_ID = PR.PRODUCT_ID;

FIRST_NAME      LAST_NAME       PRODUCT_NAME           QUANTITY
--------------- --------------- -------------------- ----------
Akshaya         Nivasini        pedigree dog food             2
Akshaya         Nivasini        cat dry food                  1
Prithish        Raj             bird seed mix                 2
kshaunish       Rajesh          fish food flakes              3
kaniska         Ram             puppy chew toy                1
Rukmani         Kuppusamy       pedigree dog food             2
Rukmani         Kuppusamy       bird seed mix                 1
Akshaya         Nivasini        pedigree dog food             2
Prithish        Raj             cat dry food                  2
kshaunish       Rajesh          bird seed mix                 2

10 rows selected.
```

2. Implement LEFT JOIN

Concept: LEFT JOIN returns every row from the left table and matching rows from the right table. If no match exists, the right-table columns are NULL.

2.1 Display all customers, including customers who have not placed orders

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME,
  4      C.LAST_NAME,
  5      O.ORDER_ID,
  6      O.ORDER_DATE,
  7      O.ORDER_STATUS
  8  FROM CUSTOMER C
  9  LEFT JOIN ORDERS O
 10      ON C.CUSTOMER_ID =
 11         CASE O.CUSTOMER_ID
 12             WHEN 'C001' THEN 'S101'
 13             WHEN 'C002' THEN 'S102'
 14             WHEN 'C003' THEN 'S103'
 15             WHEN 'C004' THEN 'S104'
 16             WHEN 'C005' THEN 'S105'
 17         END;

CUSTOMER_ID  FIRST_NAME      LAST_NAME       ORDER_ID   ORDER_DATE      ORDER_STATUS
------------ --------------- --------------- ---------- --------------- ---------------
S101         Akshaya         Nivasini        O001       01-SEP-26       Pending
S102         Prithish        Raj             O002       03-SEP-26       Shipped
S103         kshaunish       Rajesh          O003       05-SEP-26       Delivered
S104         kaniska         Ram             O004       07-SEP-26       Pending
S105         Rukmani         Kuppusamy       O005       10-SEP-26       Delivered
S101         Akshaya         Nivasini        O006       12-SEP-26       Pending
S102         Prithish        Raj             O007       13-SEP-26       Shipped
S103         kshaunish       Rajesh          O008       14-SEP-26       Delivered
S104         kaniska         Ram             O009       15-SEP-26       Pending
S105         Rukmani         Kuppusamy       O010       16-SEP-26       Shipped
S109         Geetha          Govind
S106         Vijay           Deepika
S108         Shreya          Raman
S110         Ramya           Krishnan
S107         Aura            Nivas

15 rows selected.
```

2.2 Find customers without any purchases

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME,
  4      C.LAST_NAME
  5  FROM CUSTOMER C
  6  WHERE NOT EXISTS (
  7      SELECT 1
  8      FROM ORDERS O
  9      WHERE O.CUSTOMER_ID =
 10          CASE C.CUSTOMER_ID
 11              WHEN 'S101' THEN 'C001'
 12              WHEN 'S102' THEN 'C002'
 13              WHEN 'S103' THEN 'C003'
 14              WHEN 'S104' THEN 'C004'
 15              WHEN 'S105' THEN 'C005'
 16          END
 17  );

CUSTOMER_ID  FIRST_NAME      LAST_NAME
------------ --------------- ---------------
S110         Ramya           Krishnan
S109         Geetha          Govind
S108         Shreya          Raman
S107         Aura            Nivas
S106         Vijay           Deepika
```

2.3 Display all products, including products with no sales

```
SQL> SELECT
  2      PR.PRODUCT_ID,
  3      PR.PRODUCT_NAME,
  4      OD.ORDER_ID,
  5      OD.QUANTITY
  6  FROM PRODUCT PR
  7  LEFT JOIN ORDERDETAILS OD
  8      ON PR.PRODUCT_ID = OD.PRODUCT_ID;

PRODUCT_ID   PRODUCT_NAME              ORDER_ID     QUANTITY
---------- ------------------------ ---------- ----------
P101         pedigree dog food        O0005             2
P101         pedigree dog food        O0001             2
P101         pedigree dog food        O0006             2
P102         cat dry food             O0001             1
P102         cat dry food             O0007             2
P103         bird seed mix            O0005             1
P103         bird seed mix            O0002             2
P103         bird seed mix            O0008             2
P104         fish food flakes         O0003             3
P105         puppy chew toy           O0004             1
P106         cat ball toy
P107         pet shampoo
P108         Dog collar
P109         pet vitamin supplement
P110         pet sleeping bed

15 rows selected.
```

3. Implement RIGHT JOIN

Concept: RIGHT JOIN returns every row from the right table and matching rows from the left table.

3.1 Display all orders with customer information

```
SQL> SELECT
  2      O.ORDER_ID,
  3      O.ORDER_DATE,
  4      O.ORDER_STATUS,
  5      C.CUSTOMER_ID,
  6      C.FIRST_NAME,
  7      C.LAST_NAME
  8  FROM CUSTOMER C
  9  RIGHT JOIN ORDERS O
 10      ON C.CUSTOMER_ID =
 11         CASE O.CUSTOMER_ID
 12             WHEN 'C001' THEN 'S101'
 13             WHEN 'C002' THEN 'S102'
 14             WHEN 'C003' THEN 'S103'
 15             WHEN 'C004' THEN 'S104'
 16             WHEN 'C005' THEN 'S105'
 17          END;

ORDER_ID   ORDER_DATE      ORDER_STATUS    CUSTOMER_ID  FIRST_NAME      LAST_NAME
---------- --------------- --------------- ------------ --------------- ---------------
O006       12-SEP-26       Pending         S101         Akshaya         Nivasini
O001       01-SEP-26       Pending         S101         Akshaya         Nivasini
O007       13-SEP-26       Shipped         S102         Prithish        Raj
O002       03-SEP-26       Shipped         S102         Prithish        Raj
O008       14-SEP-26       Delivered       S103         kshaunish       Rajesh
O003       05-SEP-26       Delivered       S103         kshaunish       Rajesh
O009       15-SEP-26       Pending         S104         kaniska         Ram
O004       07-SEP-26       Pending         S104         kaniska         Ram
O010       16-SEP-26       Shipped         S105         Rukmani         Kuppusamy
O005       10-SEP-26       Delivered       S105         Rukmani         Kuppusamy

10 rows selected.
```

3.2 Find orders where customer details are missing

```
SQL> SELECT
  2      O.ORDER_ID,
  3      O.CUSTOMER_ID,
  4      O.ORDER_DATE
  5  FROM ORDERS O
  6  WHERE NOT EXISTS (
  7      SELECT 1
  8      FROM CUSTOMER C
  9      WHERE C.CUSTOMER_ID = O.CUSTOMER_ID
 10  );

ORDER_ID   CUSTOMER_ID  ORDER_DATE
---------- ------------ ---------------
O001       C001         01-SEP-26
O002       C002         03-SEP-26
O003       C003         05-SEP-26
O004       C004         07-SEP-26
O005       C005         10-SEP-26
O006       C001         12-SEP-26
O007       C002         13-SEP-26
O008       C003         14-SEP-26
O009       C004         15-SEP-26
O010       C005         16-SEP-26

10 rows selected.
```

3.3 Display all payment records with order details

```
SQL> SELECT
  2      P.PAYMENT_ID,
  3      P.ORDER_ID,
  4      P.PAYMENT_MODE,
  5      P.PAYMENT_STATUS,
  6      P.TRANSACTION_AMOUNT,
  7      O.ORDER_DATE,
  8      O.ORDER_STATUS
  9  FROM ORDERS O
 10  RIGHT JOIN PAYMENT P
 11      ON O.ORDER_ID = P.ORDER_ID;

PAYMENT_ID   ORDER_ID   PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT ORDER_DATE      ORDER_STATUS
------------ ---------- --------------- --------------- ------------------ -------------- ---------------
PAY001       O001       UPI             Success                      2500 01-SEP-26      Pending
PAY002       O002       Card            Success                      1800 03-SEP-26      Shipped
PAY003       O003       Cash            Success                      1200 05-SEP-26      Delivered
PAY004       O004       UPI             Failed                       3000 07-SEP-26      Pending
PAY005       O005       Card            Success                      2200 10-SEP-26      Delivered
PAY006       O006       UPI             Success                      1500 12-SEP-26      Pending
PAY007       O007       Cash            Failed                        900 13-SEP-26      Shipped
PAY008       O008       UPI             Success                      3500 14-SEP-26      Delivered
PAY009       O009       Card            Success                      2800 15-SEP-26      Pending
PAY010       O010       UPI             Pending                      1700 16-SEP-26      Shipped

10 rows selected.
```

4. Retrieve Complete Order Details

Task: Generate a complete order report containing customer name, product name, quantity, order date, total amount, and payment status.

Complete Order Report

```
SQL> SELECT
  2      C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3      PR.PRODUCT_NAME,
  4      OD.QUANTITY,
  5      O.ORDER_DATE,
  6      O.TOTAL_AMOUNT,
  7      P.PAYMENT_STATUS
  8  FROM CUSTOMER C
  9  JOIN ORDERS O
 10      ON C.CUSTOMER_ID =
 11         CASE O.CUSTOMER_ID
 12             WHEN 'C001' THEN 'S101'
 13             WHEN 'C002' THEN 'S102'
 14             WHEN 'C003' THEN 'S103'
 15             WHEN 'C004' THEN 'S104'
 16             WHEN 'C005' THEN 'S105'
 17         END
 18  JOIN ORDERDETAILS OD
 19      ON O.ORDER_ID = OD.ORDER_ID
 20  JOIN PRODUCT PR
 21      ON OD.PRODUCT_ID = PR.PRODUCT_ID
 22  LEFT JOIN PAYMENT P
 23      ON O.ORDER_ID = P.ORDER_ID;

CUSTOMER_NAME            PRODUCT_NAME              QUANTITY ORDER_DATE      TOTAL_AMOUNT PAYMENT_STATUS
------------------------ ------------------------- -------- -------------- ------------ ---------------
Akshaya Nivasini         cat dry food                     1 01-SEP-26          1200.00 Success
Akshaya Nivasini         pedigree dog food                2 01-SEP-26          1200.00 Success
Prithish Raj             bird seed mix                    2 03-SEP-26           850.00 Success
kshaunish Rajesh         fish food flakes                 3 05-SEP-26          1500.00 Success
kaniska Ram              puppy chew toy                   1 07-SEP-26           650.00 Failed
Rukmani Kuppusamy        bird seed mix                    1 10-SEP-26          2100.00 Success
Rukmani Kuppusamy        pedigree dog food                2 10-SEP-26          2100.00 Success
Akshaya Nivasini         pedigree dog food                2 12-SEP-26          1000.00 Success
Prithish Raj             cat dry food                     2 13-SEP-26          1200.00 Failed
kshaunish Rajesh         bird seed mix                    2 14-SEP-26           850.00 Success

10 rows selected.
```

5. Display Customer Purchase History

5.1 All products purchased by a customer

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4      O.ORDER_ID,
  5      PR.PRODUCT_NAME,
  6      OD.QUANTITY,
  7      O.ORDER_DATE
  8  FROM CUSTOMER C
  9  JOIN ORDERS O
 10      ON C.CUSTOMER_ID =
 11         CASE O.CUSTOMER_ID
 12             WHEN 'C001' THEN 'S101'
 13             WHEN 'C002' THEN 'S102'
 14             WHEN 'C003' THEN 'S103'
 15             WHEN 'C004' THEN 'S104'
 16             WHEN 'C005' THEN 'S105'
 17         END
 18  JOIN ORDERDETAILS OD
 19      ON O.ORDER_ID = OD.ORDER_ID
 20  JOIN PRODUCT PR
 21      ON OD.PRODUCT_ID = PR.PRODUCT_ID
 22  WHERE C.CUSTOMER_ID = '&customer_id'
 23  ORDER BY O.ORDER_DATE DESC;
Enter value for customer_id: S101
old  22: WHERE C.CUSTOMER_ID = '&customer_id'
new  22: WHERE C.CUSTOMER_ID = 'S101'

CUSTOMER_ID  CUSTOMER_NAME            ORDER_ID   PRODUCT_NAME               QUANTITY ORDER_DATE
------------ ----------------------- ---------- ----------------------- ---------- --------------
S101         Akshaya Nivasini        O006       pedigree dog food                2 12-SEP-26
S101         Akshaya Nivasini        O001       pedigree dog food                2 01-SEP-26
S101         Akshaya Nivasini        O001       cat dry food                     1 01-SEP-26
```

5.2 Total amount spent by each customer

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4      SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  5  FROM CUSTOMER C
  6  JOIN ORDERS O
  7      ON C.CUSTOMER_ID =
  8         CASE O.CUSTOMER_ID
  9             WHEN 'C001' THEN 'S101'
 10             WHEN 'C002' THEN 'S102'
 11             WHEN 'C003' THEN 'S103'
 12             WHEN 'C004' THEN 'S104'
 13             WHEN 'C005' THEN 'S105'
 14         END
 15  GROUP BY
 16      C.CUSTOMER_ID,
 17      C.FIRST_NAME,
 18      C.LAST_NAME
 19  ORDER BY TOTAL_SPENT DESC;

CUSTOMER_ID  CUSTOMER_NAME             TOTAL_SPENT
------------ ------------------------ -----------
S105         Rukmani Kuppusamy              2750
S103         kshaunish Rajesh               2350
S101         Akshaya Nivasini               2200
S104         kaniska Ram                    2150
S102         Prithish Raj                   2050
```

5.3 Number of orders placed by each customer

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4      COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS
  5  FROM CUSTOMER C
  6  LEFT JOIN ORDERS O
  7      ON C.CUSTOMER_ID =
  8         CASE O.CUSTOMER_ID
  9             WHEN 'C001' THEN 'S101'
 10             WHEN 'C002' THEN 'S102'
 11             WHEN 'C003' THEN 'S103'
 12             WHEN 'C004' THEN 'S104'
 13             WHEN 'C005' THEN 'S105'
 14         END
 15  GROUP BY
 16      C.CUSTOMER_ID,
 17      C.FIRST_NAME,
 18      C.LAST_NAME
 19  ORDER BY NUMBER_OF_ORDERS DESC;

CUSTOMER_ID  CUSTOMER_NAME             NUMBER_OF_ORDERS
------------ ------------------------ ----------------
S104         kaniska Ram                             2
S105         Rukmani Kuppusamy                       2
S102         Prithish Raj                            2
S101         Akshaya Nivasini                        2
S103         kshaunish Rajesh                        2
S106         Vijay Deepika                           0
S110         Ramya Krishnan                          0
S108         Shreya Raman                            0
S109         Geetha Govind                           0
S107         Aura Nivas                              0

10 rows selected.
```

5.4 Latest purchase details of customers

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4      O.ORDER_ID,
  5      O.ORDER_DATE,
  6      O.TOTAL_AMOUNT,
  7      O.ORDER_STATUS
  8  FROM CUSTOMER C
  9  JOIN ORDERS O
 10      ON C.CUSTOMER_ID =
 11         CASE O.CUSTOMER_ID
 12             WHEN 'C001' THEN 'S101'
 13             WHEN 'C002' THEN 'S102'
 14             WHEN 'C003' THEN 'S103'
 15             WHEN 'C004' THEN 'S104'
 16             WHEN 'C005' THEN 'S105'
 17         END
 18  WHERE O.ORDER_DATE = (
 19      SELECT MAX(O2.ORDER_DATE)
 20      FROM ORDERS O2
 21      WHERE O2.CUSTOMER_ID =
 22          CASE C.CUSTOMER_ID
 23              WHEN 'S101' THEN 'C001'
 24              WHEN 'S102' THEN 'C002'
 25              WHEN 'S103' THEN 'C003'
 26              WHEN 'S104' THEN 'C004'
 27              WHEN 'S105' THEN 'C005'
 28          END
 29  );

CUSTOMER_ID  CUSTOMER_NAME             ORDER_ID   ORDER_DATE      TOTAL_AMOUNT ORDER_STATUS
------------ ------------------------- ---------- --------------- ------------ ---------------
S101         Akshaya Nivasini          O006       12-SEP-26            1000.00 Pending
S102         Prithish Raj              O007       13-SEP-26            1200.00 Shipped
S103         kshaunish Rajesh          O008       14-SEP-26             850.00 Delivered
S104         kaniska Ram               O009       15-SEP-26            1500.00 Pending
S105         Rukmani Kuppusamy         O010       16-SEP-26             650.00 Shipped
```

6. Generate Multi-Table Business Reports

Report 1: Customer Order Report

```
SQL> SELECT C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  2         O.ORDER_ID,
  3         O.ORDER_DATE,
  4         O.ORDER_STATUS
  5  FROM CUSTOMER C
  6  JOIN ORDERS O
  7      ON C.CUSTOMER_ID =
  8         CASE O.CUSTOMER_ID
  9             WHEN 'C001' THEN 'S101'
 10             WHEN 'C002' THEN 'S102'
 11             WHEN 'C003' THEN 'S103'
 12             WHEN 'C004' THEN 'S104'
 13             WHEN 'C005' THEN 'S105'
 14         END
 15  ORDER BY O.ORDER_DATE DESC;

CUSTOMER_NAME             ORDER_ID   ORDER_DATE      ORDER_STATUS
------------------------- ---------- --------------- --------------
Rukmani Kuppusamy         O010       16-SEP-26       Shipped
kaniska Ram               O009       15-SEP-26       Pending
kshaunish Rajesh          O008       14-SEP-26       Delivered
Prithish Raj              O007       13-SEP-26       Shipped
Akshaya Nivasini          O006       12-SEP-26       Pending
Rukmani Kuppusamy         O005       10-SEP-26       Delivered
kaniska Ram               O004       07-SEP-26       Pending
kshaunish Rajesh          O003       05-SEP-26       Delivered
Prithish Raj              O002       03-SEP-26       Shipped
Akshaya Nivasini          O001       01-SEP-26       Pending

10 rows selected.
```

Report 2: Sales Report

```
SQL> SELECT PR.PRODUCT_NAME,
  2         SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  3         SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  4  FROM PRODUCT PR
  5  JOIN ORDERDETAILS OD
  6      ON PR.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY PR.PRODUCT_ID, PR.PRODUCT_NAME
  8  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_NAME              QUANTITY_SOLD TOTAL_REVENUE
------------------------- ------------- -------------
pedigree dog food                     6          3000
bird seed mix                         5          2125
fish food flakes                      3          1500
cat dry food                          3          1400
puppy chew toy                        1           650
```

Report 3: Payment Analysis Report

```
SQL> SELECT PAYMENT_MODE,
  2         COUNT(*) AS NUMBER_OF_TRANSACTIONS,
  3         SUM(CASE
  4               WHEN UPPER(PAYMENT_STATUS) = 'SUCCESS' THEN 1
  5               ELSE 0
  6             END) AS SUCCESSFUL_PAYMENTS
  7  FROM PAYMENT
  8  GROUP BY PAYMENT_MODE
  9  ORDER BY NUMBER_OF_TRANSACTIONS DESC;

PAYMENT_MODE    NUMBER_OF_TRANSACTIONS SUCCESSFUL_PAYMENTS
--------------- ---------------------- ------------------
UPI                                  5                  3
Card                                 3                  3
Cash                                 2                  1
```

Report 4A: Top Purchasing Customers (by total items)

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3         SUM(OD.QUANTITY) AS TOTAL_ITEMS
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6      ON C.CUSTOMER_ID =
  7         CASE O.CUSTOMER_ID
  8             WHEN 'C001' THEN 'S101'
  9             WHEN 'C002' THEN 'S102'
 10             WHEN 'C003' THEN 'S103'
 11             WHEN 'C004' THEN 'S104'
 12             WHEN 'C005' THEN 'S105'
 13         END
 14  JOIN ORDERDETAILS OD
 15      ON O.ORDER_ID = OD.ORDER_ID
 16  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
 17  ORDER BY TOTAL_ITEMS DESC;

CUSTOMER_ID  CUSTOMER_NAME             TOTAL_ITEMS
------------ ------------------------ -----------
S103         kshaunish Rajesh                   5
S101         Akshaya Nivasini                   5
S102         Prithish Raj                       4
S105         Rukmani Kuppusamy                  3
S104         kaniska Ram                        1
```

Report 4B: Customers with Maximum Orders

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS ORDER_COUNT
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6      ON C.CUSTOMER_ID =
  7         CASE O.CUSTOMER_ID
  8             WHEN 'C001' THEN 'S101'
  9             WHEN 'C002' THEN 'S102'
 10             WHEN 'C003' THEN 'S103'
 11             WHEN 'C004' THEN 'S104'
 12             WHEN 'C005' THEN 'S105'
 13         END
 14  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
 15  HAVING COUNT(O.ORDER_ID) = (
 16      SELECT MAX(CNT)
 17      FROM (
 18          SELECT COUNT(*) AS CNT
 19          FROM ORDERS
 20          GROUP BY CUSTOMER_ID
 21      )
 22  );

CUSTOMER_ID  CUSTOMER_NAME             ORDER_COUNT
------------ ------------------------ -----------
S104         kaniska Ram                        2
S101         Akshaya Nivasini                   2
S102         Prithish Raj                       2
S105         Rukmani Kuppusamy                  2
S103         kshaunish Rajesh                   2
```

Report 4C: Customers with Highest Spending

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6      ON C.CUSTOMER_ID =
  7         CASE O.CUSTOMER_ID
  8             WHEN 'C001' THEN 'S101'
  9             WHEN 'C002' THEN 'S102'
 10             WHEN 'C003' THEN 'S103'
 11             WHEN 'C004' THEN 'S104'
 12             WHEN 'C005' THEN 'S105'
 13         END
 14  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
 15  HAVING SUM(O.TOTAL_AMOUNT) = (
 16      SELECT MAX(SPENT)
 17      FROM (
 18          SELECT SUM(TOTAL_AMOUNT) AS SPENT
 19          FROM ORDERS
 20          GROUP BY CUSTOMER_ID
 21      )
 22  );

CUSTOMER_ID  CUSTOMER_NAME             TOTAL_SPENT
------------ ------------------------ -----------
S105         Rukmani Kuppusamy               2750
```

7. Conclusion

Thus, INNER JOIN, LEFT JOIN, and RIGHT JOIN were applied to the Pet Shop Management System. The queries combine customer, order, product, and payment information to produce purchase histories, order reports, sales reports, and payment analysis.